

Red de Nodos IoT Distribuidos con ESP32 y LoRa para Monitoreo Ambiental

Resumen. Se plantea un sistema IoT de redes de sensores distribuidos en zonas protegidas, donde cada nodo basado en ESP32 mide condiciones ambientales (temperatura, humedad, presión, calidad del aire, etc.) y las transmite vía LoRa a un nodo central cercano al acceso de Internet. El nodo principal actúa como *gateway*: recibe datos de los nodos remotos y los envía a la nube (p. ej. servidor web o plataforma IoT), además de reenviar comandos de control a los sensores. Esta arquitectura aprovecha las largas distancias y bajo consumo de LoRa para cubrir áreas amplias, combinando topologías en estrella y multisalto (mesh) según convenga. En modo normal se emplea una malla preconfigurada (definida por el usuario), y en modo “emergencia” los nodos intentan reenviar datos de forma dinámica a través de vecinos cercanos hasta alcanzar el nodo principal (reenvío *multihop*). A continuación se detallan el estado del arte, selección de sensores y antenas, protocolos y seguridad, estructura de software y ejemplo de código para cada nodo.

1. Introducción

Las redes de sensores inalámbricos (WSN) permiten el monitoreo en tiempo real de variables ambientales críticas en zonas remotas o protegidas. En particular, usar ESP32 con módulos LoRa extiende el alcance hasta kilómetros sin gran consumo energético. A diferencia de una comunicación Wi-Fi local, LoRa en bandas ISM (915/868 MHz) brinda comunicación de largo alcance (varios km en campo libre). Sin embargo, LoRa suele usarse en topología estrella (LoRaWAN) con un gateway central. Proponemos un diseño híbrido que combine modos de conexión personalizados: el nodo principal (cerca de Internet) recibe datos de nodos hijos mediante rutas predefinidas o dinámicas (*mesh*), garantizando cobertura incluso en entornos con obstáculos.

El sistema incluye: (a) **Nodos sensores** distribuidos (basados en ESP32+LoRa) que adquieren métricas ambientales, y (b) un **Nodo**

principal (ESP32 o similar con LoRa y acceso a Internet) que consolida la información en la nube y distribuye comandos. Cada nodo sensor está alimentado por batería (posiblemente con panel solar) y contiene un módem LoRa (p. ej. SX1276) conectado a sensores ambientales. El nodo principal, cercano a la fuente de Internet, podría montar una antena de mayor ganancia para cobertura extensa, así como WiFi/4G para conectarse al servidor. Se implementará un protocolo de enrutamiento que permita configurar rutas de entrega personalizadas entre nodos (p. ej. en estrella, cadena o malla completa, según esquemas indicados por el usuario) y un mecanismo de reenvío alternativo en caso de falla (*modo emergencia*). Los detalles se exponen en las siguientes secciones.

2. Estado del arte y motivación

Las tecnologías LPWAN como LoRa/LoRaWAN son ideales para monitoreo ambiental remoto debido a su largo alcance y bajo consumo. En contexto de IoT, varios estudios han usado nodos LoRa para medir clima, calidad de aire o incluso estados de colmenas. Por ejemplo, Valencia-Barahona et al. (2024) diseñaron una red LoRa con 2 nodos sensores y un *gateway*, midiendo humedad, temperatura y peso de colmenas. También se han propuesto arquitecturas híbridas que combinan topologías en estrella con mallas dinámicas para LoRaWAN. García et al. (2025) demostraron que LoRaWAN convencional alcanza ≈ 5 km en urbano y ≈ 20 km en rural, pero el multisalto extiende el alcance efectivo más allá del rango de la pasarela. Añadir nodos repetidores o usar rutas multihop mejora cobertura y confiabilidad.

Por otro lado, existen librerías emergentes para LoRa mesh. Por ejemplo, **LoRaMesher** permite armar redes mesh de nodos LoRa (ESP32) sin una *gateway* central, usando un protocolo de vector de distancia para enrutar paquetes nodo a nodo. Esto ilustra que es factible implementar reenvío multisalto con ESP32+LoRa. El reto principal es diseñar un

software que administre dinámicamente la topología (configuración manual o automática en modo emergencia) y garantice la seguridad de los datos (cifrado y autenticación) en la comunicación LoRa.

3. Diseño de hardware: sensores y antenas

Cada nodo sensor incluirá los siguientes componentes clave:

- **Microcontrolador:** ESP32 (o variante LPWAN como TTGO/Heltec con radio SX1276) para CPU integrada, WiFi/Bluetooth y control de sensores.
- **Módem LoRa:** Integrado en la placa o módulo SX127x para comunicación sub-GHz de largo alcance.
- **Sensores ambientales:** Para monitorear el estado ambiental de la zona protegida. Entre los más comunes figuran:
 - *Temperatura y humedad:* por ejemplo sensores digitales SHT31/BME280/BME680 (Bosch) o DHT22 (menos preciso). Miden temperatura ambiental ($\pm 0.01^\circ\text{C}$) y humedad relativa.
 - *Presión atmosférica:* integrado en BME280/BME680, útil para cálculos meteorológicos.
 - *Calidad del aire/gases:* MQ-135 (multi-gas) es popular para CO_2 , NH_3 , NO_x y VOCs; también sensores como MiCS-2714 (CO), MiCS-4514 (NO_x), o CCS811/SGP30 (eCO_2 /TVOC). Permiten cuantificar contaminantes locales.
 - *Partículas en suspensión ($\text{PM}_{2.5}$):* sensores ópticos SDS011 o PMS7003 para monitorear material particulado fino.
 - *Parámetros adicionales:* opcionalmente sensor de radiación UV (influye en ozono) o medidores de luz/noise.
- **Antenas:** Críticas para el alcance LoRa. Se recomienda usar antenas externas para la banda escogida (p. ej. 915 MHz en América). Los tipos incluyen:
 - *Antenas omnidireccionales (dipolo/colineal):* cubren 360° horizontal, con ganancia moderada ($\sim 2\text{--}9\text{ dBi}$). Colineales de fibra de vidrio (tipo “stick”) proporcionan amplio alcance exterior con resistencia al clima.
 - *Antenas direccionales (Yagi, panel):* concentran señal en una dirección,

aumentando el alcance puntual. Útiles si la línea de vista es obstruida y se conoce la dirección del receptor.

- *Antenas helicoidales o stump:* compactas, integrables en dispositivos con espacio limitado.
- *Ganancias típicas:* desde $\sim 3\text{ dBi}$ (pequeñas omni) hasta $>12\text{ dBi}$ (Yagi direccional).

Los prototipos de nodos incluyen un ESP32 conectado a sensores ambientales, montado en una placa de pruebas. En la imagen de ejemplo (arriba) se observa un ESP32 (derecha) con una tira de LEDs indicadores en una **proto-board**. A estos nodos se les conectan sensores como termómetro, higrómetro, barómetro o sensores de gas. Cada nodo debe alimentarse con batería (o panel solar) y el módulo LoRa transmite los datos recopilados al nodo central. Como en estudios previos, se suelen usar módulos SX1276/78 (868/915 MHz) que junto a una antena adecuada permiten comunicar a varios km.

En terreno se instalarán estaciones meteorológicas con anemómetros, veletas y sensores ambientales. Por ejemplo, la fotografía ilustra un anemómetro típico montado en zona rural. Estos nodos de campo miden **temperatura, humedad relativa y presión atmosférica** – parámetros que influyen directamente en la dispersión de contaminantes. Además, se integran sensores de partículas (p.ej. $\text{PM}_{2.5}$) y de gases contaminantes (NO_2 , SO_2 , O_3 , CO , CO_2 , VOCs) para medir la **calidad del aire**. Por ejemplo, sensores como MQ-135 son económicos y detectan gases múltiples (NH_3 , CO_2 , compuestos orgánicos). Los datos raw de estos sensores (digital o analógico) son leídos periódicamente por el ESP32 y empaquetados para envío por LoRa.

Especificaciones de antenas

La selección de antenas depende del despliegue. Para distancias largas, se prefieren antenas de alta ganancia. Según la literatura, antenas de 915 MHz adecuadas para LoRa permiten cubrir distancias de **10–15 km** en campo libre, comparables a frecuencias inferiores (433 MHz) con menos restricciones regulatorias. La antena es crucial para el alcance y estabilidad de la red. Se usará una antena **omnidirigida** de $\sim 5\text{--}9\text{ dBi}$ en nodos

con cobertura horizontal, o direccional (~9–13 dBi) en nodos identificados con punto fijo, mejorando la relación señal-ruido a grandes distancias. En todos los casos se respetan patrones de polarización verticales y estándares de impedancia (50 Ω).

4. Protocolos de comunicación y seguridad

4.1. Protocolo de radio

La red emplea el **físico LoRa** (Chirp Spread Spectrum) en la banda ISM seleccionada (p. ej. 915 MHz en América Latina). LoRa ofrece tasas bajas (unos cientos de bps) pero excepcional rango. El ajuste de parámetros (Spreading Factor, ancho de banda, codificación) se elegirá para balancear alcance y latencia: por ejemplo, SF=9–11 y BW=125 kHz proporcionan largas distancias con bajo consumo. En modo normal se puede usar **LoRa P2P**, donde cada nodo envía paquetes al siguiente salto o directamente al nodo central. También se podría implementar LoRaWAN (con TTN o servidor privado), pero dado que se exige reenvío multihop personalizado, se opta por un esquema meshed ad-hoc. Una librería útil es LoRaMesher, que permite establecer rutas dinámicas entre nodos LoRa sin gateway fijo.

Las rutas preconfiguradas (estrella o cadena) se definirán en cada nodo (por ejemplo, indicando los *IDs* de los vecinos autorizados). En el *modo emergencia*, si un nodo no puede alcanzar su enlace habitual (p. ej. nodo intermedio caído), cambia a modo “relay”: busca nodos vecinos cercanos y reenvía sus datos hasta llegar al nodo principal (un enrutamiento por inundación leve). Esto implementa de hecho una topología *mesh* flexible, mejorando la resiliencia de la red.

4.2. Protocolos de red y transporte

Para la lógica interna, se empleará un esquema simple: cada paquete contiene el ID del nodo emisor, la métrica ambiental (por ejemplo CSV con temp, hum, etc.) y un posible comando descendente. El nodo principal interpreta estos mensajes y puede responder con paquetes de comando (p.ej. activar actuadores, cambiar frecuencia de muestreo). En la capa superior, el nodo principal puede conectarse por WiFi (o 4G) a un servidor o plataforma IoT (p.ej. ThingsBoard, MQTT broker, web custom) para visualizar datos y

enviar comandos de control remoto. Así, el ESP32 gateway puede usar MQTT sobre TCP/IP para comunicarse con la nube.

4.3. Seguridad

La protección de datos es crítica en IoT ambiental, aunque la información no es extremadamente sensible, se recomienda cifrar los paquetes. En LoRaWAN estándar el cifrado AES-128 está incorporado de fábrica. En nuestro diseño P2P se implementará cifrado simétrico adicional: por ejemplo, se puede usar **AES-128** en modo CBC/CTR y HMAC-SHA256 para autenticidad de cada mensaje. Un método típico es aplicar *encrypt-then-MAC*: primero cifrar el payload con AES (clave compartida) y luego calcular HMAC con SHA-256 para detectar adulteraciones. El microcontrolador ESP32 puede usar la librería ArduinoCrypto o similar. Esto asegura confidencialidad e integridad en el enlace LoRa. Además, cada nodo debe autenticarse (por ejemplo, mediante IDs o claves únicas precompartidas) al unirse a la red. El firmware incluirá gestión de claves en memoria protegida. En el enlace a Internet (WiFi-MQTT), se usarán TLS/SSL para cifrar la sesión con el servidor final.

5. Ejemplo de implementación y código

En la práctica se usará Arduino IDE o PlatformIO con las librerías **LoRa** (semtech) y **WiFiClient**. A continuación se bosquejan ejemplos de código (muy simplificados) para el nodo principal y un nodo hijo.

None

```
// Nodo principal (ESP32 LoRa Gateway)
```

```
#include <LoRa.h>
```

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
const long freq = 915E6;
```

```
const char* ssid = "mi-red";
```

```
const char* password = "xxxx";
```

```
const char* serverURL =  
"http://mi-servidor.com/api/upload";
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  LoRa.begin(freq);
```

```
  // Configurar WiFi
```

```
  WiFi.begin(ssid, password);
```

```

while (WiFi.status() != WL_CONNECTED)
delay(500);
Serial.println("Conectado a WiFi");
}

void loop() {
// Recibir datos de nodos hijos
int packetSize = LoRa.parsePacket();
if (packetSize) {
String data = "";
while (LoRa.available()) {
data += (char)LoRa.read();
}
Serial.printf("Recibido via LoRa: %s\n",
data.c_str());
// Enviar datos a servidor web o base de
datos IoT
if (WiFi.status() == WL_CONNECTED) {
HTTPClient http;
http.begin(serverURL);
http.addHeader("Content-Type",
"application/json");
String json = "{\"data\":\"" + data + "\"}";
int httpCode = http.POST(json);
http.end();
}
}
// Por seguridad podríamos enviar
comandos periódicos o chequeos
// ...
}

```

cpp
Copiar

None

```

// Nodo hijo (ESP32 sensor LoRa)
#include <LoRa.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

const long freq = 915E6;
Adafruit_BME280 bme;

void setup() {
Serial.begin(115200);
LoRa.begin(freq);
if (!bme.begin(0x76)) {
Serial.println("No se encuentra BME280");
}
}

void loop() {
// Leer sensores
float temp = bme.readTemperature();
float hum = bme.readHumidity();

```

```

float pres = bme.readPressure() / 100.0F;
// Empaquetar datos (JSON o CSV)
String payload = "NODO1," +
String(temp,1) + "," + String(hum,1) + "," +
String(pres,1);
// Enviar por LoRa
LoRa.beginPacket();
LoRa.print(payload);
LoRa.endPacket();

Serial.printf("Enviado: %s\n",
payload.c_str());
delay(60000); // enviar cada 1 minuto
}

```

Estos ejemplos ilustran la idea: cada nodo hijo envía sus lecturas periódicamente, y el gateway las reenvía a la plataforma. En un proyecto real se añadirían rutinas de cifrado (AES) antes de `LoRa.beginPacket()` y de `LoRa.read()`, así como manejo de reintentos y *timeouts*. También se programarían los modos de *mesh* en caso de que falle la ruta usual (p.ej. intentando reenviar por otro vecino).

6. Estructura propuesta del repositorio

El proyecto debe organizarse de forma modular. Una posible estructura de directorios es:

python
Copiar

None

```

/ (raíz del proyecto)
├── docs/ # Documentación
general (paper, especificaciones, esquemas)
│   ├── paper_Proyecto.pdf
│   └── README.md
├── src/ # Código fuente para
nodos y gateway
│   ├── main_node/ # Firmware del
nodo principal (ESP32 Gateway)
│   │   ├── main.ino
│   │   ├── config.h
│   │   └── libs/ # Librerías adicionales
si es necesario
│   └── sensor_node/ # Firmware de los
nodos sensores hijos
│       ├── sensor.ino
│       ├── config.h
│       └── libs/
└── hardware/ # Diagramas de
hardware, fotos, lista de componentes

```

```

|   |   | sensors/      # Datasheets de
|   |   | sensores (BME280, SDS011, MQ-135, etc.)
|   |   | |   |   | pcb/      # Esquemas de montaje
|   |   | |   |   | / protoboard
|   |   | |   |   | |   |   | antenas/      # Especificaciones de
|   |   | |   |   | |   |   | antenas usadas
|   |   | |   |   | |   |   | web/          # Código y config del
|   |   | |   |   | |   |   | servidor / dashboard web
|   |   | |   |   | |   |   | |   |   | dashboard.html
|   |   | |   |   | |   |   | |   |   | server_app.py
|   |   | |   |   | |   |   | |   |   | mqtt_config.yaml
|   |   | |   |   | |   |   | |   |   | README.md      # Descripción del
|   |   | |   |   | |   |   | |   |   | proyecto
|   |   | |   |   | |   |   | |   |   | LICENSE        # Licencia de uso

```

- **docs/paper_Proyecto.pdf:** Documento técnico (paper) con metodología y resultados.
- **src/main_node/** contiene el firmware del nodo principal (receptor LoRa y módulo WiFi). El archivo main.ino incluye la lógica de recepción LoRa y conexión a Internet. Pueden usarse archivos de configuración (config.h) para parámetros (SSID, frecuencia, claves).
- **src/sensor_node/** contiene el firmware de los nodos hijos. En sensor.ino se leen los sensores (BME280, MQ-135, etc.) y se envían datos por LoRa. También se incluye el código para escuchar comandos de control desde el gateway.
- **hardware/** guarda documentos de diseño: diagramas de conexiones (ej. protoboard o PCB con sensor y ESP32), así como especificaciones o «bolsa de piezas» (BOM) de los sensores y antenas seleccionados.
- **web/** alberga el código del servidor web o dashboard (por ejemplo, un script Python que reciba las lecturas vía HTTP o MQTT, y una página HTML/JS para visualizar las métricas y enviar comandos). Esto ilustra el «página principal» desde la cual se monitorea la red y se interactúa con los nodos.
- **README.md** en la raíz resume el objetivo del proyecto, las instrucciones de instalación y uso, y referencias a la documentación.
- Esta estructura modular facilita el mantenimiento del proyecto: cada nodo tiene su código independiente, la documentación técnica está centralizada en docs/, y la parte de servidor/dashboard (si existe) está separada en web/.

7. Conclusiones

El proyecto propuesto diseña una red IoT mesh de largo alcance basada en ESP32 y

LoRa para vigilancia ambiental en zonas protegidas. Al combinar modos de conexión configurables y ruteo de emergencia, se asegura cobertura en entornos difíciles. Los sensores seleccionados (temperatura, humedad, presión, gases, partículas) cubren las principales variables medioambientales. Las antenas de alta ganancia permiten enlaces de varios kilómetros. En cuanto a software, la implementación requiere programar tanto el lado LoRa (con rutinas de envío/recepción y cifrado AES/HMAC) como la parte de servidor web. La estructura de repositorio propuesta garantiza un proyecto organizado. En conjunto, esta solución permite monitorear en tiempo real el impacto de la contaminación y fenómenos meteorológicos en áreas remotas, ampliando el alcance más allá de lo que lograría una red convencional Wi-Fi o 4G solo.

Fuentes: Se han consultado estudios y guías técnicas sobre redes LoRaWAN y WSN, además de revisiones de sensores MQ-135 y documentación comercial sobre parámetros ambientales. Estas referencias fundamentan la selección de hardware y protocolos aquí propuestos.